SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

SIMATS ENGINEERING (SAVEETHASCHOOLOFENGINEERING)

Department of Computer Science & Engineering

CO2 - ASSESSMENT TOOL -3

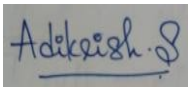
Error Analysis Task

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO2**	**Assessment**	**Assessment Tool 3: Error Analysis Task**
Weight age	**20 % of CIA**	**Total Marks**	**100**
CO2Statement	***Design effective test cases using black-box, white-box testing techniques and to control methods flow***		
Student Name	**Balaguru S**	**Reg.No.**	**192312070**
Section/Batch	**Slot A / 2023**	**Date**	**04-11-2026**

Code of Conduct:

I, Balaguru S (192312070), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment .I understands that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



3. A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.

Error Analysis of the Control Flow Graph and Path Testing Approach in a White Box Testing Task.

1.Identify the errors in the control flow graph (CFG) and in the path testing approach.

2.Analyse the root causes of the inadequate test coverage.

3.Construct a proper control flow representation and identify all independent paths.

4.Justify how improved path testing ensures better coverage and defect detection.

1. Problem Understanding and Scope

A developer has tested a module with white box testing but drew an incomplete control flow graph. Decision nodes and loops are missing, so the developer believed far fewer paths existed than the code really contains and stopped testing early. This report finds the errors in the graph and in the path testing approach, traces each to its root cause, rebuilds the CFG, derives all independent paths, and shows with measured evidence that the corrected approach improves coverage and defect detection.

Assumptions. The question supplies no source code, so a small representative module, processBatch, is analysed. It is a batch-payment routine with every construct named in the question: a loop, a chain of decisions, a decision after the loop and two exits. Rules: amounts <= 0 are rejected; amounts above limit are rejected; amounts above balance are rejected; valid amounts are debited; after the loop an alert is raised if rejected transactions outnumber accepted ones; the function returns the final balance, or −1 when there is nothing to process. All path traces, coverage figures and fault-detection results below come from executing the module with the listed data.

1.1 Module under test

Line	Statement	CFG node
1	int processBatch(int n, int balance, int amt[], int limit) {	entry
2	int i = 0, ok = 0, rej = 0;	1
3	if (n <= 0) return -1;	2 (predicate), 3
4	while (i < n) {	4 (predicate)
5	if (amt[i] <= 0) {	5 (predicate)
6	rej++;	6
7	} else if (amt[i] > limit) {	7 (predicate)
8	rej++;	8
9	} else if (amt[i] <= balance) {	9 (predicate)

Line	Statement	CFG node
10	balance -= amt[i]; ok++;	10
11	} else { rej++; }	11
12	i++; }	12 (join node)
13	if (rej > ok) alert();	13 (predicate), 14
14	return balance; }	15

2. The Developer's Approach and Its Measured Coverage

Figure 1 reproduces the graph the developer drew. It has five nodes and one decision (n <= 0). The whole while loop and the three-way if / else if / else chain are squeezed into a single block (node 4), and the post-loop test rej > ok with its alert() call does not appear at all. From this graph the developer concluded that there were only two paths and designed two tests: **T1** n = 0, giving -1; and **T2** n = 1, amt = [50], balance = 100, limit = 100, giving 50. Both paths were executed, so "100% path coverage" was reported.

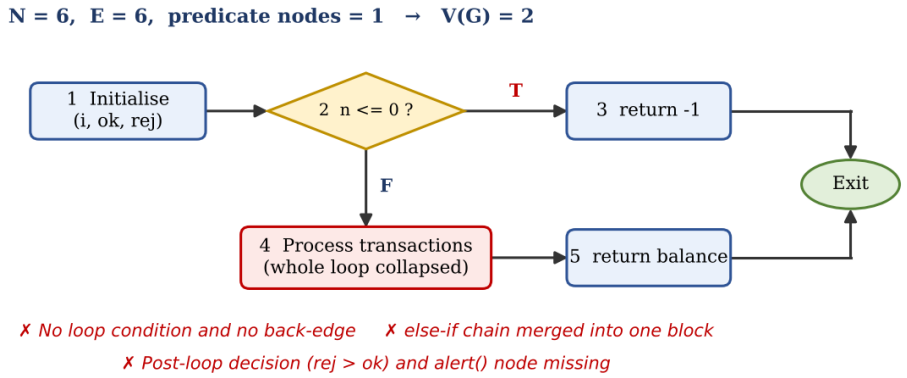


Figure 1: The developer's incomplete control flow graph (as drawn)

That claim does not survive a check against the real code (Section 5 gives the correct graph). The two tests execute only 11 of the 15 code blocks, 8 of the 12 decision outcomes and 2 of the 7 independent paths. The four blocks never run are exactly the rejection and alert logic: nodes 6, 8, 11 and 14.

3. Error Identification

Comparing the developer's graph and test plan with the code in Section 1 gives seven errors. E1 to E5 are errors in the graph; E6 and E7 are errors in the path testing approach built on it.

ID	Error found	What the code actually contains	Consequence for testing
E1	**Loop missing.** No condition node for while (i < n) and no back-edge; the loop body is one block.	Predicate node 4 and the back-edge 12 → 4. The body runs 1, 2 or many times.	Iteration counts are never varied; faults that appear only from the second pass stay hidden.
E2	**Else-if chain collapsed.** Three decisions (lines 5, 7, 9) are missing.	Predicate nodes 5, 7, 9 and four outcome blocks 6, 8, 10, 11.	Invalid-amount, over-limit and insufficient-funds paths are never executed.
E3	**Post-loop decision missing.** rej > ok and alert() are absent.	Predicate node 13 and block 14 before the final return.	Alert behaviour and the rej = ok boundary are untested.
E4	**Join node missing.** The merge point i++ is not drawn.	Join node 12 receives edges from 6, 8, 10 and 11 and returns to 4.	Without a join the loop cannot be drawn and four edges are lost.
E5	**Wrong size and complexity.** N = 6, E = 6, one predicate, V(G) = 2.	N = 16, E = 21, six predicates, V(G) = 7.	Test effort is under-estimated by 5 of 7 paths (71%).
E6	**Coverage claimed from the wrong graph.** "100% path coverage" means 2 of 2 paths of the sketch.	Only 2 of 7 independent paths, 8 of 12 branches and 11 of 15 blocks are executed.	A false sense of adequacy ends testing too early.
E7	**Weak path-testing practice.** Only the happy path with one iteration; no path predicates, no feasibility check, no check of side effects.	State (balance, ok, rej) carried between iterations and the alert() side effect both matter.	Defects in state carry-over and in the alert logic go unseen.

Measure	Developer's CFG	Correct CFG	Gap
Nodes (including Exit)	6	16	10 missing
Predicate (decision) nodes	1	6	5 missing

Measure	Developer's CFG	Correct CFG	Gap
Edges	6	21	15 missing
Loop back-edges	0	1	1 missing
Cyclomatic complexity V(G)	2	7	5 independent paths unplanned

4. Root Cause Analysis

A why-chain shows how the main error arose. The loop was missing because the graph was drawn from the idea "process every transaction" and not from the code; no fixed procedure forced one node per condition; the CFG was treated as documentation and not as the tool that drives path and test-data selection; and no V(G) cross-check or coverage measurement existed to expose the gap. Six root causes result.

ID	Root cause	Technical reasoning	Errors caused
RC 1	**Graph drawn from a functional summary, not from the code**	A while is a predicate plus a back-edge. Read as "process the batch" it looks like one activity, so abstraction removes the control structure that path testing needs.	E1, E3
RC 2	**Misunderstanding of what a node and a predicate are**	Every two-outcome condition is its own predicate node. An else-if chain of k conditions gives k nodes and k + 1 outcome blocks, not one; treating it as one decision hides two predicates.	E2, E3
RC 3	**No formal CFG construction procedure**	Without fixed steps (number lines, form blocks, mark predicates, add join nodes and edges, add one exit) the merge node i++ is forgotten and the loop is never closed.	E4
RC 4	**No complexity cross-check**	V(G) = P + 1 and V(G) = E − N + 2 must agree with the code. A value of 2 for a 14-line function with a loop and six conditions is an obvious warning that was never examined.	E5
RC 5	**Confusing coverage metrics**	Executing all paths of an incomplete graph is not path coverage of the program. Statement (73.3%) and branch (66.7%) coverage were never measured; no coverage tool or review was used.	E6

ID	Root cause	Technical reasoning	Errors caused
RC 6	**Test data chosen by intuition, not derived from path predicates**	Tests were picked for "normal" behaviour. Path predicates (for example amt[i] <= 0, then rej > ok) were not solved, loop cases (0, 1, 2, many iterations) were ignored, and the aim was the fewest tests.	E7

RC1 to RC4 remove five of the six predicates, so no test is planned for five of the seven independent paths; RC5 and RC6 turn that blind spot into a confident but wrong report of adequate coverage.

5. Corrected Control Flow Representation

The graph was rebuilt with a fixed procedure: (1) number the lines; (2) group straight-line statements into basic blocks; (3) give every condition its own predicate node; (4) add a join node wherever branches merge; (5) close every loop with a back-edge to its condition; (6) merge all returns into one Exit node; (7) check V(G) three ways. Figure 2 shows the finished graph.

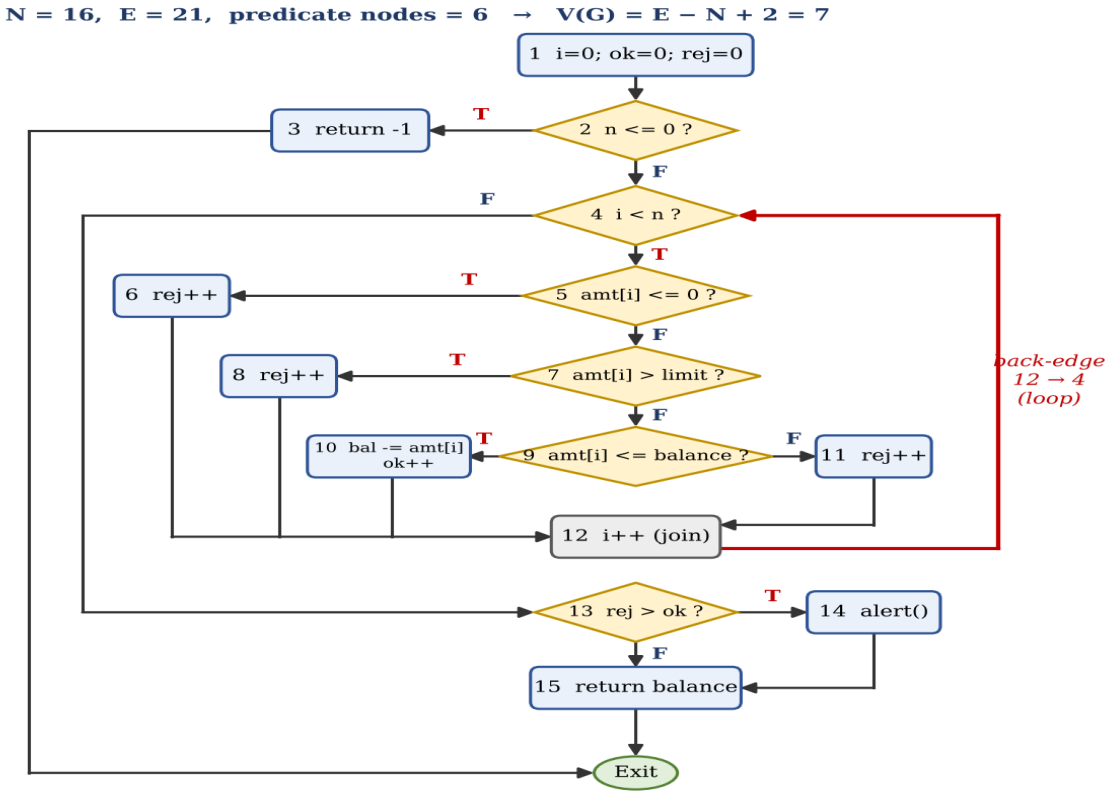


Figure 2: Corrected control flow graph of processBatch (diamonds = predicates, red edge = loop back-edge)

Edge list (21): 1→2, 2→3, 2→4, 3→Exit, 4→5, 4→13, 5→6, 5→7, 6→12, 7→8, 7→9, 8→12, 9→10, 9→11, 10→12, 11→12, 12→4, 13→14, 13→15, 14→15, 15→Exit.

Method	Calculation	V(G)
Predicate count	P + 1 = 6 + 1 (nodes 2, 4, 5, 7, 9, 13)	7
Edges and nodes	E − N + 2 = 21 − 16 + 2	7
Regions	6 enclosed regions + 1 outer region	7

All three methods agree, so the graph is consistent with the code. V(G) = 7 is also below the usual limit of 10, so the module is acceptable in complexity but needs seven tests, not two.

6. Independent Paths and Test Cases

A basis set of seven linearly independent paths was chosen, each adding at least one edge or edge combination not used by the others. Test data were derived by solving the decision outcomes along each path, and each path was dry-run on the code to confirm that it is feasible and gives the stated result.

Path	Node sequence	Decision outcomes	Test data (n; amt[]; balance; limit)	Expected result
P1	1-2-3-Exit	2=T	0; [ ]; 100; 100	-1, no alert
P2	1-2-4-5-7-9-10-12-4-13-15-Exit	2=F, 5=F, 7=F, 9=T, 13=F	1; [50]; 100; 100	50, no alert
P3	1-2-4-5-6-12-4-13-14-15-Exit	5=T, 13=T	1; [0]; 100; 100	100, alert()
P4	1-2-4-5-7-8-12-4-13-14-15-Exit	5=F, 7=T, 13=T	1; [500]; 1000; 100	1000, alert()
P5	1-2-4-5-7-9-11-12-4-13-14-15-Exit	7=F, 9=F, 13=T	1; [80]; 50; 100	50, alert()
P6	1-2-4-5-7-9-10-12-4-5-7-9-10-12-4-13-15-Exit	two valid iterations, 13=F	2; [30, 40]; 100; 100	30, no alert
P7	1-2-4-5-6-12-4-5-7-9-10-12-4-13-15-Exit	reject then accept, 13=F	2; [0, 50]; 100; 100	50, no alert (rej = ok = 1)

Independence check. P1 to P5 already touch all 21 edges (full branch coverage) but have rank 5, below V(G) = 7. P6 is the first path that traverses the back-edge twice and P7 the first where a rejected and an accepted transaction alternate; with them the rank is 7 (checked on the 7 × 21 edge-incidence matrix). **Infeasible path.** The path 1-2-4-13-15 (loop skipped) exists in the graph but is impossible, since n > 0 and i = 0 force node 4 to be true on first entry; it is excluded. **Loop cases.** Zero iterations are covered by P1, one by P2 to P5, two by P6 and P7; n = max and max + 1 can be added from BVA.

7. Coverage and Defect Detection After Correction

Metric	Developer's 2 tests	Corrected 7 tests
Statement coverage (blocks)	11 / 15 = 73.3%	15 / 15 = 100%
Branch coverage (decision outcomes)	8 / 12 = 66.7%	12 / 12 = 100%
Edge coverage	13 / 21 = 61.9%	21 / 21 = 100%
Independent paths executed	2 / 7 = 28.6%	7 / 7 = 100%
Loop iterations exercised	1 only	0, 1 and 2
Seeded faults detected (below)	1 / 6 = 16.7%	6 / 6 = 100%

To measure defect detection, six faults of the kind a missing CFG hides were seeded one at a time into the module, and both test sets were run against each faulty version.

ID	Seeded fault	Found by developer's tests?	Found by corrected tests
M1	i++ skipped on the rejection branches, so the loop never ends	No	P3, P4, P5, P7 (loop does not terminate)
M2	rej >= ok coded instead of rej > ok	No	P7 only (rej = ok = 1 raises a false alert)
M3	rej++ missing in the insufficient-funds branch (node 11)	No	P5 (no alert raised, expected alert)
M4	Over-limit branch (node 8) also debits the balance	No	P4 (returns 500 instead of 1000)
M5	Stale balance reused on each iteration	No	P6 (returns 60 instead of 30)
M6	n < 0 coded instead of n <= 0	Yes (T1)	P1 (returns 100 instead of -1)

Faults M1 to M5 sit exactly in the structure that the flawed graph left out: the loop, the else-if branches and the post-loop decision. The developer's tests found only M6, while the seven basis-path tests found all six.

8. Justification of the Improved Path Testing Approach

Reason	Explanation	Evidence in this report
Structural completeness	Every block, decision outcome and loop edge exists in the graph, so each can be targeted. A path missing from the graph can never be tested.	15/15 blocks, 12/12 branches, 21/21 edges
Linear independence	V(G) is the size of a basis set: seven tests execute every edge and each adds new behaviour. Two tests cannot.	Rank 7 for P1 to P7; developer's rank 2
Loop integrity	The back-edge allows tests for 0, 1 and several iterations; faults that depend on state carried between iterations appear only from the second pass.	M1 and M5 found by P3 to P7
Condition combinations	Mixed outcomes (one rejected, one accepted) reach the boundary of the post-loop decision.	M2 found only by P7
Manageable cost	Each iteration has four outcomes, so exhaustive testing needs 4^n paths (over a million for n = 10); the basis set stays at seven tests.	7 tests instead of 4^n
Traceability and measurement	Each test maps to a path and nodes; coverage is measured, not assumed.	Sections 6 and 7 tables

Limits and complements. Basis-path testing cannot reveal missing requirements, uninitialised variables or wrong boundary constants, because a path can run correctly with bad data; it should be combined with Boundary Value Analysis (amt = limit, limit + 1, balance, balance + 1) and data-flow testing. **Preventive checklist:** build the CFG from the code, not the description; one node per condition; one join node per merge; one back-edge per loop; one exit node; confirm V(G) by three methods; one test per basis path with a feasibility check; and report coverage from a tool, never from the sketch.

9. Conclusion

The developer's graph had five nodes and one decision where the code has fifteen blocks and six decisions. The loop, the else-if chain, the join node and the post-loop test were missing, so V(G) was reported as 2 instead of 7 and "100% path coverage" was really 28.6% of the independent paths. The root causes were drawing from a description, misunderstanding predicates, having no construction procedure or complexity check, confusing coverage metrics and choosing data by intuition. The corrected CFG (N = 16, E = 21, V(G) = 7) yields seven independent paths whose tests raise statement, branch and path coverage to 100% and fault detection from 1 of 6 to 6 of 6.

10. Mapping of This Answer to the Rubrics

Rubric criterion	Weightage	Where addressed
Error Identification	20%	Sections 2 and 3: errors E1 to E7, Figure 1, quantified gap table
Root Cause Analysis	30%	Section 4: why-chain and root causes RC1 to RC6 linked to each error
Correction & Justification	25%	Sections 5 to 8: corrected CFG, V(G), seven independent paths, fault seeding, justification
Application of Concepts	15%	CFG construction, cyclomatic complexity, basis paths, infeasible path, loop testing, coverage metrics
Clarity & Presentation	10%	Figures 1 and 2, structured tables, step-by-step sections

Rubrics for calculation:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors	
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors	
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided	
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts	
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	